

Complexité algorithmique

Term NSI — Octobre 2026

Algorithmes de recherche séquentiels et dichotomiques, complexité algorithmique, notation de Landau $O(g(n))$: $O(n)$, $O(\log n)$,...

1 COURS

1.1 Pourquoi mesurer la complexité ?

Pour un même problème, plusieurs algorithmes (ou implémentations) sont possibles. Pour les comparer **indépendamment de la machine**, on ne mesure pas un temps en secondes : on **compte le nombre d'opérations** effectuées, en fonction de la **taille n des données** (souvent $n = \text{len}(L)$).

On note $T(n)$ ce nombre d'opérations. C'est la **complexité temporelle**.

La *complexité spatiale* mesure de la même façon la mémoire utilisée. Sauf mention contraire, « complexité » désigne la complexité temporelle.

1.2 Calculer $T(n)$

Convention : une instruction simple compte pour **1** (affectation, calcul, comparaison, accès `L[i]`, `append`, `return`...). Ainsi `a = b`, `a = b * c` et `a = a + b * c` coûtent chacune 1. On ne compte pas la gestion du compteur de boucle (initialisation, test d'arrêt, incrémentation).

Souvent, on ne compte qu'**un seul type d'opération significative** (par exemple les comparaisons pour une recherche). Le résultat final en $O(\dots)$ est le même.

Structure	Coût
Instructions à la suite	on additionne les coûts
Boucle	nombre d'itérations × coût du corps de la boucle
<code>if ... else</code>	coût du test + coût de la branche la plus coûteuse
Appel de fonction	coût de la fonction appelée (attention : pas forcément 1 !)

Boucle `while` : pour trouver le nombre d'itérations, on cherche un **variant de boucle** : une quantité entière positive qui **décroît strictement** à chaque tour. Il prouve que la boucle se termine, et ses valeurs initiale et finale donnent le nombre de tours.

Exemple :

```

1  def somme(L):
2      s = 0                # 1
3      for x in L:          # n itérations
4          s = s + x        # 1 par itération
5      return s            # 1

```

$$T(n) = 1 + n \times 1 + 1 = n + 2$$

Même calcul avec une boucle `while` :

```

1  def somme_while(L):
2      s = 0                # 1
3      i = 0                # gestion du compteur : non compté
4      while i < len(L):    # variant : len(L) - i
5          s = s + L[i]     # 1 par itération
6          i = i + 1        # gestion du compteur : non compté
7      return s            # 1

```

Le variant de boucle `len(L) - i` vaut n au départ, diminue de 1 à chaque tour et vaut 0 à la sortie : la boucle s'exécute **n fois**.

$$T(n) = 1 + n \times 1 + 1 = n + 2$$

On retrouve le même $T(n)$ qu'avec la boucle `for` : le type de boucle ne change pas la complexité.

1.3 Meilleur cas, pire cas

Pour une même taille n , le nombre d'opérations peut dépendre des données. Pour la recherche séquentielle d'un élément dans une liste de taille n :

- **meilleur cas** : l'élément est en première position $\rightarrow$ 1 comparaison ;
- **pire cas** : l'élément est absent $\rightarrow$ n comparaisons.

Sauf précision contraire, on étudie toujours le pire des cas : c'est une garantie.

1.4 Notation O (notation de Landau)

On s'intéresse au comportement de $T(n)$ pour **n très grand**. Pour obtenir la classe de complexité $O(g(n))$ à partir de $T(n)$:

1. on garde uniquement le **terme qui croît le plus vite** ;
2. on supprime les **constantes multiplicatives**.

Exemples : $T(n) = n + 2 \Rightarrow O(n)$; $T(n) = 3n^2 + 5n + 2 \Rightarrow O(n^2)$; $T(n) = 7 \Rightarrow O(1)$.

1.5 Principales classes de complexité

Classées de la plus rapide à la plus lente :

Classe	Nom	Exemple	Opérations pour $n = 1000$	Si n double, le temps...
$O(1)$	constante	accès à <code>L[i]</code>	1	ne change pas
$O(\log n)$	logarithmique	recherche dichotomique	≈ 10	augmente de 1 étape
$O(n)$	linéaire	recherche séquentielle	1 000	double
$O(n \log n)$	quasi-linéaire	tri fusion	$\approx 10\,000$	double (un peu plus)
$O(n^2)$	quadratique	deux boucles imbriquées	1 000 000	est multiplié par 4
$O(n^3)$	cubique	trois boucles imbriquées	1 000 000 000	est multiplié par 8
$O(2^n)$	exponentielle	tester toutes les combinaisons	$\approx 10^{301}$	est élevé au carré

Exemple : deux boucles imbriquées. La fonction suivante compte les couples `(L[i], L[j])` dont la somme vaut `cible`.

```

1  def compte_paires(L, cible):
2      nb = 0                                # 1
3      for i in range(len(L)):                # n itérations
4          for j in range(len(L)):            # n itérations pour chaque i
5              if L[i] + L[j] == cible:        # 1 (test)
6                  nb = nb + 1                 # 1 (pire cas : test toujours vrai)
7      return nb                              # 1

```

- Le corps de la boucle interne coûte au pire 2 opérations.
- La boucle interne fait n tours : elle coûte $n \times 2 = 2n$.
- La boucle externe répète n fois la boucle interne : $n \times 2n = 2n^2$.

$$T(n) = 1 + 2n^2 + 1 = 2n^2 + 2$$

On garde le terme dominant $2n^2$ et on supprime la constante 2 : la complexité est $O(n^2)$.

À retenir : deux boucles imbriquées de n tours chacune donnent une complexité **quadratique** $O(n^2)$.

1.6 Outils mathématiques

Somme des entiers consécutifs (boucles imbriquées dont la boucle interne dépend de la boucle externe) :

$$1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$$

Logarithme en base 2 : $\log_2(n)$ est le **nombre de divisions par 2** nécessaires pour passer de n à 1. Autrement dit, si $n = 2^k$, alors $\log_2(n) = k$.

Exemple : $1024 \rightarrow 512 \rightarrow \dots \rightarrow 2 \rightarrow 1$ en 10 divisions, donc $\log_2(1024) = 10$.

À la calculatrice : $\log_2(x) = \frac{\ln(x)}{\ln(2)}$.

1.7 Bilan sur les algorithmes de recherche (pire cas)

Algorithme	Condition	Complexité
Recherche séquentielle	aucune	$O(n)$
Recherche dichotomique	liste triée	$O(\log n)$

À chaque itération de la recherche dichotomique, la zone de recherche est divisée par 2 (variant de boucle : `d - g`). Le nombre d'itérations est donc de l'ordre de $\log_2(n)$ (voir exercice 6).

2 EXERCICES

Les exercices sont classés par difficulté croissante : ★ (application directe), ★★ (raisonnement), ★★★ (défi).

2.1 Exercice 1 ★ : Compter les opérations

Voici trois implémentations du même algorithme, qui construit la table de multiplication de `b` :

```

1  def multiplie1(b, n):
2      L = []
3      for i in range(n):
4          L.append(b * i)
5      return L
6
7  def multiplie2(b, n):
8      L = []
9      for i in range(n):
10         y = b * i
11         L.append(y)
12     return L
13
14 def multiplie3(b, n):
15     L = []
16     i = n - 1
17     while i >= 0:
18         y = b * i
19         L.append(y)
20         i = i - 1
21     return L

```

Règle : opérations significatives

Une opération est **significative** si elle agit sur les données ou sur le résultat. Chacune compte pour **1** :

- la création ou l'affectation d'une variable de calcul (`L = []`, `y = b * i`) ;
- un calcul ou une comparaison portant sur les données ;
- un ajout dans une liste (`L.append(...)`) ;
- le `return`.

Une ligne qui combine plusieurs calculs compte pour 1 : `L.append(b * i)` compte **1**, pas 2.

Ne sont pas significatives les instructions qui servent seulement à gérer le compteur de boucle : la ligne `for i in range(n)`, l'initialisation `i = n - 1`, le test `i >= 0` et la mise à jour `i = i - 1`.

1. Calculer $T(n)$ pour `multiplie1` et pour `multiplie2`, en appliquant cette règle.
2. `multiplie2` effectue presque deux fois plus d'opérations que `multiplie1`. Ont-elles pourtant la même classe de complexité ? Laquelle ?
3. Pour `multiplie3` : quel est le variant de boucle ? Quelles sont ses valeurs initiale et finale ? En déduire le nombre d'itérations, puis $T(n)$ et la classe de complexité.
4. Conclure : la classe de complexité dépend-elle du type de boucle (`for` ou `while`) ?

2.2 Exercice 2 ★ : Meilleur cas et pire cas

On ne compte ici que les **comparaisons** `elt == x`.

```

1  def est_present(x, L):
2      for elt in L:
3          if elt == x:
4              return True
5      return False
6
7  def compte(x, L):
8      nb = 0
9      for elt in L:
10         if elt == x:
11             nb = nb + 1
12     return nb

```

1. Pour `est_present` avec une liste de taille n : combien de comparaisons dans le meilleur cas ? Dans le pire cas ? Décrire une liste et une valeur `x` correspondant à chacun de ces cas.
2. Même question pour `compte`. Qu'est-ce qui change, et pourquoi ?
3. Donner la complexité dans le pire cas de chacune des deux fonctions.
4. Laquelle de ces deux fonctions correspond à la recherche séquentielle du TP ? Combien de comparaisons fait-elle au maximum dans une liste d'un million d'éléments ?

2.3 Exercice 3 ★★ : Déplacer le serpent dans Snake

On donne une fonction de l'interface du serpent dans le jeu *Snake* :

```

1  def supprime_queue(S):
2      # décale toutes les valeurs de S d'un cran vers la gauche,
3      # puis supprime le dernier élément
4      for i in range(len(S) - 1):
5          S[i] = S[i + 1]
6      S.pop()

```

1. Déterminer $T(n)$, le nombre d'opérations effectuées pour un serpent de taille n .
2. En déduire la classe de complexité de cette fonction.
3. La fonction est appelée à chaque image du jeu. Si le serpent passe de 50 à 100 cases, comment évolue le temps d'exécution de `supprime_queue` ?
4. En Python, `S.pop(0)` produit le même résultat que `supprime_queue(S)` en une seule instruction. Est-ce pour autant une opération en $O(1)$? Justifier.

2.4 Exercice 4 ★★ : Boucles imbriquées

```

1  def table(n):
2      T = []
3      for i in range(n):
4          for j in range(n):
5              T.append(i * j)
6      return T

```

1. Combien de fois l'instruction `T.append(i * j)` est-elle exécutée ? En déduire la complexité de `table`.
On cherche maintenant si une liste contient deux fois la même valeur. On ne compte que les comparaisons `L[i] == L[j]`.

```

1  def contient_doublon(L):
2      n = len(L)
3      for i in range(n):
4          for j in range(i + 1, n):
5              if L[i] == L[j]:
6                  return True
7      return False

```

2. Quel est le pire cas pour cette fonction ?
3. Pour une liste de taille $n = 5$, compléter le tableau suivant dans le pire cas :

i	valeurs prises par j	nombre de comparaisons
0	1, 2, 3, 4	4
1		
2		
3		
4		
Total		

4. Généraliser : exprimer le nombre total de comparaisons pour une liste de taille n sous forme d'une somme, puis le calculer à l'aide de la formule du cours.
5. En déduire la complexité de `contient_doublon`. Effectue-t-elle plus ou moins d'opérations que `table` ? Sont-elles dans la même classe de complexité ?

2.5 Exercice 5 ★★ : Fonctions qui appellent des fonctions

On ne compte que les **additions**.

```

1  def truc(n):
2      res = 0
3      for i in range(n):
4          res = res + 1
5      return res
6
7  def chose(n):
8      res = 0
9      for i in range(n):
10         res = res + truc(i)
11     return res

```

1. Déterminer $T(n)$ et la complexité de `truc`. Que renvoie `truc(n)` ?
2. Dans `chose(n)`, combien d'additions coûte l'appel `truc(i)` lorsque i vaut 0 ? 1 ? 2 ? $n - 1$?
3. En déduire $T(n)$ pour `chose` (penser à la formule du cours), puis sa complexité.
4. Que renvoie `chose(n)` ? Écrire une fonction `chose_rapide(n)` qui renvoie le même résultat en $O(1)$.

2.6 Exercice 6 ★★ : Recherche dichotomique et logarithme

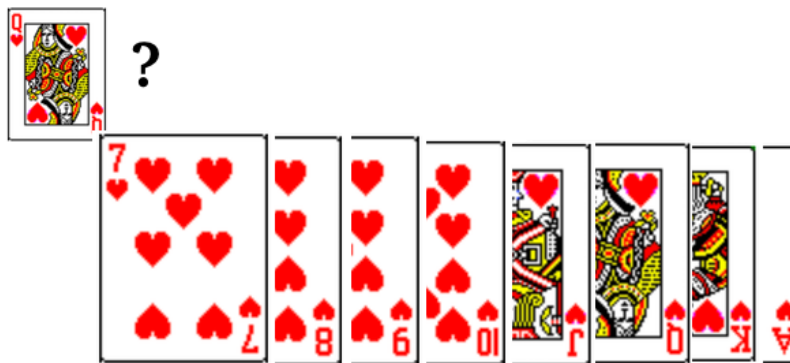


Fig. 1. – cartes triées

Les cartes sont triées dans l'ordre : 7, 8, 9, 10, J (Valet), Q (Dame), K (Roi), A (As).

1. Écrire la liste `L` représentant ce jeu de cartes : le 7 est représenté par l'entier 1, le 8 par 2, etc., jusqu'à l'As qui vaut 8. Quel entier représente la Dame Q ?
2. Compléter le tableau de suivi de la recherche dichotomique de la Dame Q :

g	d	m	L[m]	trouve
0	7	3	4	False

3. Faire de même pour la recherche de la valeur 9 (carte absente : c'est le pire cas). Combien d'itérations sont nécessaires ? Combien de comparaisons ferait une recherche séquentielle dans ce même cas ?
4. Combien de fois successives faut-il diviser 8 par 2 pour arriver à 1 ? Même question pour 16, puis pour 32. Comparer avec $\log_2(8)$, $\log_2(16)$ et $\log_2(32)$.
5. En déduire la complexité de la recherche dichotomique.
6. Compléter le tableau (nombre de comparaisons dans le pire cas, ordre de grandeur) :

Taille n de la liste	Recherche séquentielle	Recherche dichotomique
8		
1 000		
1 000 000		
1 000 000 000		

7. Le dictionnaire `ods.txt` du TP contient plusieurs centaines de milliers de mots. Quelle méthode de recherche faut-il privilégier ? Quelle condition doit vérifier la liste de mots pour pouvoir l'utiliser ?

2.7 Exercice 7 ★★★ : Défi

Déterminer la complexité des fonctions suivantes, en nombre d'additions et de soustractions. Donner d'abord la valeur exacte de $T(n)$, puis la classe de complexité.

```

1  def truc(n):
2      res = 0
3      for i in range(n):
4          res = res + 1
5      return res
6
7  def machin(n):
8      res = truc(n)
9      for i in range(n):
10         res = res - 1
11     return res
12
13 def chose(n):
14     res = 0
15     for i in range(n):
16         res = res + machin(i)
17     return res
18
19 def fonction_finale(n):
20     res = []
21     for i in range(n):
22         res.append(chose(n))
23     return res

```

Bonus : que renvoie `fonction_finale(n)` ? Proposer une version beaucoup plus efficace.